

CORE JAVA

1) String vs StringBuffer vs StringBuilder

String

String is **immutable**, meaning once the object is created, its value **cannot be changed**. Any modification creates a **new object in memory**. Strings are stored in the **String Constant Pool** for memory optimization.

StringBuffer

StringBuffer is **mutable** and **thread-safe**. It allows modification of the same object and uses **synchronized methods**, which makes it safe for **multi-threaded environments**, but slower.

StringBuilder

StringBuilder is **mutable but NOT thread-safe**. It does not use synchronization, making it **faster** than StringBuffer, and is preferred in **single-threaded applications**.

How to explain to panel

“If immutability and security are required, I use String.

If thread safety is required, I use StringBuffer.

If performance is important and the application is single-threaded, I use StringBuilder.”

2) == vs equals()

== operator

The == operator **compares memory references**, not actual content. It checks whether **both variables point to the same object** in memory.

equals() method

equals() compares the **actual content/value** of objects. In classes like String, equals() is **overridden** to compare character values instead of references.

How to explain to panel

“== checks reference equality, whereas equals() checks logical equality.

For object content comparison, we should always use equals().”

3) Object Class vs Wrapper Classes

Object Class

Object is the **root class** of Java. Every class implicitly extends Object. It provides common methods like equals(), hashCode(), toString(), clone().

Wrapper Classes

Wrapper classes (Integer, Double, Boolean, etc.) convert **primitive data types into objects**, allowing primitives to be used in **Collections and APIs** that work only with objects.

How to explain to panel

“Object is the parent of all classes, while wrapper classes allow primitives to behave like objects, especially useful in collections and frameworks.”

4) Abstract Class vs Interface

Abstract Class

An abstract class can have **abstract and non-abstract methods, instance variables, and constructors**. It supports **partial abstraction** and allows method implementations.

Interface

An interface provides **100% abstraction** (before Java 8). It supports **multiple inheritance**, and methods are public and abstract by default (Java 8 allows default methods).

How to explain to panel

“If I need shared behavior with state, I use an abstract class.
If I need multiple inheritance and pure abstraction, I use an interface.”

5) Method Overloading vs Method Overriding

Method Overloading

Overloading happens **within the same class**, where methods have the **same name but different parameters**. It is **compile-time polymorphism**.

Method Overriding

Overriding occurs in **parent-child relationship**, where child class provides its own implementation of a method. It is **runtime polymorphism**.

How to explain to panel

“Overloading improves readability at compile time, while overriding provides dynamic behavior at runtime.”

6) Checked vs Unchecked Exception

Checked Exception

Checked exceptions are verified at **compile time**. They must be either **handled using try-catch** or **declared using throws** (e.g., IOException).

Unchecked Exception

Unchecked exceptions occur at **runtime** and extend RuntimeException (e.g., NullPointerException). They are usually caused by **programming mistakes**.

How to explain to panel

“Checked exceptions represent recoverable conditions, while unchecked exceptions represent programming errors.”

7) Array vs ArrayList

Array

Array is a **fixed-size** data structure that can store **primitive or object values**. It is faster but lacks flexibility.

ArrayList

ArrayList is a **dynamic array** that can grow or shrink automatically. It stores only **objects** and provides rich APIs.

How to explain to panel

“If size is fixed and performance is critical, I use arrays.
If dynamic resizing is required, I use ArrayList.”

8) HashMap vs Hashtable

HashMap

HashMap is **not synchronized**, allows **one null key and multiple null values**, and is **faster**.

Hashtable

Hashtable is **synchronized**, does not allow null keys or values, and is **thread-safe but slower**.

How to explain to panel

“In modern applications, HashMap is preferred, and for concurrency we use ConcurrentHashMap instead of Hashtable.”

9) Thread vs Runnable

Thread

Thread is a **class**. Since Java doesn't support multiple inheritance, extending Thread limits flexibility.

Runnable

Runnable is a **functional interface**, allowing a class to extend another class while still supporting multithreading.

How to explain to panel

“Runnable is preferred because it supports better object-oriented design and separation of task from execution.”

10) Stack Memory vs Heap Memory

Stack Memory

Stores **local variables and method calls**. Memory is allocated per thread and deallocated automatically.

Heap Memory

Stores **objects and instance variables**. It is shared across threads and managed by the **Garbage Collector**.

How to explain to panel

“Stack is thread-safe and short-lived, heap is shared and used for object storage.”

IMPORTANT QUESTIONS

1. String vs StringBuffer vs StringBuilder
2. String literal vs new String()
3. == vs equals()
4. Object class vs Wrapper classes
5. Primitive vs Wrapper class
6. Autoboxing vs Unboxing
7. Immutable vs Mutable objects
8. Shallow copy vs Deep copy
9. Cloneable vs Serializable